INDIAN INSTITUTE OF INFORMATION TECHNOLOGY ALLAHABAD

Department of Information Technology

Cyber Security (IT-30001) | Autumn Semester 2026

GROUP 4 — ASSIGNMENT 4 REPORT

Web Server Security: Vulnerability Demonstration and Mitigation

Instructor: Dr. Soumyadev Maity

Vulnerability Ownership

Student Name	Roll Number	Vulnerability Performed	Variant
Aryan Srivastava	IIT2024501	Cross-Site Request Forgery (CSRF)	GET-based CSRF via hidden `<img>` tag
Agam Ghotra	IIT2024226	Cross-Site Scripting (XSS)	Reflected XSS
Chandan Sapkale	IIT2024258	OS Command Injection	In-band command injection
Soham Donode	IIT2024259	SQL Injection	UNION-based data extraction (Variant 2)

Submission date: 27 October 2026

This combined report consolidates the four individual implementations submitted by Group 4. Each section describes the application, vulnerable behavior, demonstration, root cause and impact, implemented mitigation, additional defensive measures, and testing. All demonstrations are intended for the locally developed educational applications only.

1. Reflected Cross-Site Scripting (XSS) — Agam Ghotra (IIT2024226)

1.1 Overview and Objectives

This implementation demonstrates reflected Cross-Site Scripting (XSS) in a locally developed educational search page built with Node.js and Express. The application provides vulnera-

ble and mitigated modes. In vulnerable mode, the query supplied through the HTTP request is inserted into the returned HTML without encoding; in mitigated mode, HTML-significant characters are escaped before rendering. The objective is to identify the unsafe rendering path, demonstrate a harmless proof of concept, and verify that output encoding prevents the same input from executing.

1.2 Application Architecture and Vulnerable Implementation

The browser submits search text through the `q` query parameter to the `GET /` route in `app.js`. The `mode` parameter selects the vulnerable or safe rendering path. In vulnerable mode, raw query text is inserted into the HTML result area; in safe mode, `escapeHtml(q)` is used. The form input value is escaped in both modes, while the deliberately vulnerable sink is the result-area insertion.

```
const q = typeof req.query.q === 'string' ? req.query.q : '';
const vulnerable = req.query.mode === 'vulnerable';
const rendered = vulnerable ? q : escapeHtml(q);
```

1.3 Exploit Demonstration and Observations

The local lab uses the harmless proof-of-concept payload below. In vulnerable mode, the server reflects the query into the HTML response and the browser interprets the script element, displaying an alert. Ordinary text is displayed normally and is not by itself proof of XSS. In safe mode, the same payload is displayed as literal text and no alert appears.

```
<script>alert('Reflected XSS demo')</script>
```

1.4 Root Cause and Potential Impact

The root cause is insertion of untrusted query-string input into an HTML response without context-appropriate output encoding. The browser parses the response as HTML, so attacker-controlled markup can become active content. In a real application, reflected XSS could enable page manipulation, phishing content, actions performed in the victim's browser context, or access to data exposed to page scripts. The impact depends on application privileges, authentication design, browser protections and client-side data. The demonstration payload only displays an alert and does not access or transmit user data.

1.5 Mitigation Implemented

The implemented mitigation is HTML output encoding through `escapeHtml()` before the query is inserted into the result area. The function converts HTML-significant characters to en-

tities: ampersand to `&`, less-than to `<`, greater-than to `>`, quotation mark to `"`, and apostrophe to `'`. The browser therefore receives encoded text instead of a literal script tag, preventing the demonstrated payload from executing.

1.6 Additional Mitigations and Testing

Additional protections include framework auto-escaping, a maintained allow-list HTML sanitizer when rich HTML is intentionally supported, a restrictive Content Security Policy, HttpOnly cookies to limit JavaScript access to cookies, and safe DOM APIs such as `textContent` instead of `innerHTML` or `document.write`. These controls complement rather than replace output encoding. Testing should compare ordinary text, the harmless payload in vulnerable mode, and the same payload in safe mode on the local application at `127.0.0.1`. Evidence should be captured from actual execution; screenshots should not be fabricated.

1.7 Conclusion

The lab confirms that reflecting untrusted input without encoding allows the browser to interpret data as active markup. Context-appropriate HTML entity encoding makes the demonstrated input inert. Evidence should show the alert in vulnerable mode and literal-text rendering in safe mode.

2. In-Band OS Command Injection — Chandan Sapkale (IIT2024258)

2.1 Overview and Objectives

This implementation demonstrates in-band OS command injection through a simulated Windows Ping utility built with Node.js and Express. Command output is returned by the server in the HTTP response and displayed in a terminal-style area. The vulnerable endpoint constructs a shell command and passes it to `exec()`, while the secure endpoint validates the target and invokes `ping.exe` directly with `execFile()`.

2.2 Application Architecture and Vulnerable Implementation

The browser frontend in `index.html` sends a JSON POST request containing a `target` field. The Express backend exposes `POST /ping_vulnerable` and `POST /ping_secure`. The vulnerable route reads `req.body.target` without an allow-list, appends it to a command string and passes it to `child_process.exec()`, which invokes a shell on Windows. Timeout and output-buffer limits constrain some execution behavior but do not prevent shell interpretation.

```
var target = req.body.target;
var command = "ping -n 3 " + target;
```

2.3 Exploit Demonstration and Observations

The representative local-lab input is shown below. In Windows `cmd.exe`, the ampersand is a command separator. When appended to the ping command and passed to `exec()`, the shell can interpret it as the intended ping command followed by an additional command. The callback returns output as JSON, allowing output from the additional command to appear in the browser. Other patterns in the lab illustrate pipe (`|`), conditional AND (`&&`), network-information and environment-output behavior; these examples are for the isolated lab only.

```
127.0.0.1 & whoami
```

2.4 Root Cause and Potential Impact

The root cause is concatenation of untrusted input into a command string combined with execution through a shell. Shell metacharacters may therefore be interpreted as control syntax, and the vulnerable endpoint does not restrict the target to an expected hostname or IP-address character set. In a real deployment, the application process could execute unintended commands with its own privileges, potentially exposing configuration, network details, environment variables or files available to the process.

2.5 Mitigation Implemented

The secure endpoint applies allow-list validation and uses `execFile()` with a separate argument array and `shell: false`. The allow-list `/^[a-zA-Z0-9.\-]{1,253}$/` accepts only letters, digits, dots and hyphens within the specified length. Inputs containing characters such as ampersands, pipes, spaces, semicolons, quotes or brackets are rejected with HTTP 400 before process execution. Valid targets are passed as an argument to `ping.exe` rather than concatenated into a shell command.

```
execFile("ping.exe", ["-n", "3", target],
         { shell: false, timeout: 10000, maxBuffer: 64 * 1024 },
         callback)
```

2.6 Additional Mitigations and Testing

Additional measures include avoiding shell invocation, passing arguments as separate array elements, validating input against the expected format, and running the web server under a dedicated least-privilege account. The frontend uses `textContent` to display command output, which avoids interpreting output as HTML; this is separate from the server-side command-injection fix. Testing should confirm normal ping output, vulnerable endpoint behavior, rejec-

tion of the injection input with HTTP 400 by the secure endpoint, handling of empty input, and display of network errors.

2.7 Conclusion

The demonstration shows how command construction from untrusted input can let shell metacharacters alter execution. Allow-list validation and direct execution with `execFile()` and `shell: false` remove the shell interpretation path used by the demonstrated payload.

3. SQL Injection — UNION-Based Data Extraction — Soham Donode (IIT2024259)

3.1 Overview and Objectives

This implementation demonstrates Variant 2 of SQL injection, UNION-based data extraction, in TechMart, a demo e-commerce application with a Python Flask backend and SQLite database. The product search feature is the attack point. A mode toggle switches the `/search` endpoint between a vulnerable string-concatenated query and a mitigated parameterized query so the same input can be tested against both implementations.

3.2 Application and Database Architecture

The public `products` table is accessed through the search feature. A separate `users` table stores username, password, role and credit-card fields and is not linked from the user interface. The vulnerable `/search` route obtains `q` from the request and concatenates it into a `LIKE` query. Since user input becomes part of the SQL statement itself, SQL syntax in the search term can alter the query structure.

```
query = ("SELECT id, name, category, price, description "
         "FROM products WHERE name LIKE '%" + q + "%'")
```

3.3 Exploit Demonstration and Observations

In VULNERABLE mode, the following payload was submitted through the product search field. The injected `UNION SELECT` supplies five values to match the original query's five selected columns. The trailing SQL comment removes the remainder of the original statement, and values from the `users` table are mapped into columns displayed in the product results. The observed result was three additional rows from the `users` table, including administrator username and password data, returned through an unauthenticated search page.

```
xyz' UNION SELECT id, username, role, 0, password FROM users-- -
```

3.4 Root Cause and Potential Impact

The root cause is direct concatenation of untrusted search input into SQL rather than binding it as data. Because result columns are predictable and rendered to the page, a UNION query with a matching column count can combine the product query with a query against another table. In a real application, this could expose sensitive records not intended for the product search, depending on database permissions and the privileges of the application account.

3.5 Mitigation Implemented

The `/search` route was changed to use a parameterized query. The SQL statement remains fixed and the search term is bound as a parameter, with wildcard characters included in the bound value. The database driver treats the payload as data rather than reparsing it as SQL syntax. When the identical payload was submitted in SECURE mode, the application returned "No products matched your search," and no `users`-table data was returned.

```
query = "SELECT id, name, category, price, description FROM products WHERE
    name LIKE ?"
cur = db.execute(query, ("%" + q + "%",))
```

3.6 Additional Mitigations and Testing

Additional defensive measures include allow-list input validation as defense in depth, least-privilege database permissions that restrict the application account to necessary tables, and generic client-facing error messages while detailed errors are logged server-side. Testing should demonstrate that the payload extracts rows only in vulnerable mode and is treated as an ordinary non-matching search string in secure mode.

3.7 Conclusion

The lab demonstrates how UNION-based SQL injection can expose data from a table unrelated to the intended product search. Parameterized queries preserve query structure and prevent the supplied search string from being interpreted as SQL syntax.

4. GET-Based Cross-Site Request Forgery (CSRF) — Aryan Srivastava (IIT2024501)

4.1 Objective and Setup

This implementation demonstrates GET-based Cross-Site Request Forgery using two locally hosted websites: a legitimate profile application at `http://localhost:3000` and a separate attacker page at `http://localhost:3001`. The objective is to show that a browser may automatically include a session cookie with a cross-site request, allowing a sensitive action to be triggered without the user knowingly submitting it, and then demonstrate a server-side mitigation.

4.2 Application Architecture and Vulnerable Implementation

The profile application provides login, profile display and a change-email API. After login, the server creates a session cookie. The vulnerable endpoint accepts a GET request with a valid session and updates the email using the supplied `email` query parameter, without checking whether the request originated from the legitimate site. The session cookie configuration does not enable `SameSite=Strict`, and the login flow does not use a CSRF token.

```
const session = await getSession();
const newEmail = searchParams.get("email");
await updateSession({ email: newEmail });
```

4.3 Exploit Demonstration and Observations

After the user logs into the profile app, the user visits an attacker-controlled page styled as a fake prize page. That page contains a hidden zero-pixel image whose source points to the change-email endpoint. Loading the image triggers a GET request, and the browser may attach the session cookie. The server then treats the request as authenticated and changes the profile email to `hacked@attacker.com` without a deliberate action by the user. The submitted report records the initial profile email as `aryan@google.com` and the post-attack email as `hacked@attacker.com`.

```
<img src="http://localhost:3000/api/change-email?email=hacked@attacker.com"
     width="0" height="0" style="display:none" />
```

4.4 Root Cause and Potential Impact

The root cause is a state-changing endpoint that accepts a cross-site GET request and relies on the session cookie as its only authorization check. Browsers can automatically attach cookies to requests initiated by other pages, depending on cookie policy and browser behavior. In a real application, CSRF could cause an authenticated user's account settings or other state to be changed without informed consent. The demonstrated effect is a profile email change.

4.5 Mitigation Implemented

The implementation checks request origin information before processing the change. It reads `Sec-Fetch-Site` and uses `Origin`, or `Referer` as a fallback, to compare the source host with the request host. Cross-site requests are rejected with HTTP 403 and a JSON error. The report also identifies `SameSite=Strict` on the session cookie as a complementary option. State-changing operations should preferably avoid GET and use CSRF tokens where appropriate.

```
const fetchSite = request.headers.get("sec-fetch-site");
const source = request.headers.get("origin") ?? request.headers.get("referer
    ");
const sameOrigin = source
  ? new URL(source).host === new URL(request.url).host
  : false;
if (fetchSite ? fetchSite !== "same-origin" : !sameOrigin) {
  return NextResponse.json({ error: "cross site request blocked" },
                           { status: 403 });
}
```

4.6 Additional Mitigations

Other CSRF defenses include synchronizer tokens, which require a secret token tied to the user's session and validated on state-changing requests; SameSite cookies, which restrict when browsers attach session cookies to cross-site requests; re-authentication for sensitive actions, which requires the user to confirm important changes; and avoiding state-changing GET requests, because GET requests can be triggered by links, images, and other embedded resources. These measures reduce the chance that a victim's authenticated browser can be induced to perform an unintended action.

Testing should record the user logged into the profile application, the attacker page being opened, and the profile after the request. In the vulnerable run, the email should change to the attacker-controlled address. After enabling the request-origin check, the same cross-site request should be rejected and the email should remain unchanged. The test should be performed only

against the locally developed application.

4.7 Testing and Conclusion

See the testing procedure described in the preceding subsection: the vulnerable run changes the profile email to the attacker-controlled address, while the same cross-site request is rejected with HTTP 403 once the request-origin check is enabled.

5. Overall Conclusion

Group 4 implemented and documented four web security vulnerabilities: reflected XSS, in-band OS command injection, UNION-based SQL injection, and GET-based CSRF. Each demonstration isolates a different trust-boundary failure: unencoded user input entering HTML, user input entering a shell command, user input altering SQL structure, and a browser-authenticated request changing state without origin validation. The mitigations are specific to each context: HTML output encoding, allow-list validation with shell-free process execution, parameterized SQL queries, and cross-site request validation with appropriate cookie protections. All testing should remain within the locally developed educational applications, and evidence should be captured from actual runs.